上板测试逐步实施指南（达芬奇-lite + AD9238 + 本工程）

> 目标：**照着做一遍就能复现。**每一步都给出「操作 / 期望现象 / 不符时怎么办」。 前置阅读：`docs/04_wiring_checklist.md`（接线表与上电检查）、 `docs/05_vivado_sim_impl_report.md`（设计与验证全貌）。

---

0. 你要准备的东西

类别	具体	关键参数
开发板	正点原子 达芬奇-lite（XC7A35TFGG484-2）	—
载板	HSEDA 双 AD9238 标准版	40 针排母
下载器	板载 FT2232（JTAG + USB 串口共用一根 USB 线）	—
信号源	**双通道**任意波形/函数发生器，**两路独立单端输出、共地**	CH1→载板 `J1` / `CH1`，CH2→载板 `J2` / `CH2`，**各 10 Vpp / 高阻 / 0 V 偏置 / 两路相差 90°**（详见 §1.3）
BNC→SMA 同轴电缆	2 根，50 Ω	**★ 不要用杜邦线接模拟输入**（详见 §1.3）
示波器	≥2 通道，≥100 MHz	探 `ad_clk`、`a_out`、`z_out`、`u_out`
万用表	任意	上电前查极性与跳线
杜邦线	24 根（数据）+ 6 根（时钟/输出）	**建议 ≤ 5 cm**，最好用排线
上位机	笔记本 + USB 线（板载 FT2232 同时给 JTAG 和串口）	装好 USB 串口驱动，`Python` 可跑 `tools/uart_scope.py`

> **★ 信号源必须设 10 Vpp / 高阻。**载板输入端是 AD8065 → AD8138 的 ±5 V 衰减链， 量程对应 ±5 V。若设成 1 Vpp，`s_n` / `c_n` 只有几十个 LSB，UART 波形里就是一条噪声带。
>
> ⚠️ **ILA 现在已经可用**（Vivado 许可证已升级到 **GOLD**，但必须显式设 `XILINXD_LICENSE_FILE`， 否则会命中同目录下的 BASIC 许可 —— 见 §2.2）。 所以本指南的内部信号观测**全部走 UART 遥测**（2 Mbaud，18 字节定长帧）， 只有「时钟波形 / ABZ / UVW 边沿」这几项用示波器。这已经足够完成 §8 的全部验收项。

---

1. 断电接线（约 20 分钟）

1.1 把载板插到 `J6`

达芬奇-lite 上有两个 2×26 排针：`J6`（Bank 16）与 `J5`（Bank 15）。

- 载板 40 针排母 **插** `J6`， `pin 1` 对 `pin 1`。
- `J6-1` 是 `+5V`、 `J6-3` 是 `VCCIO_B16`。
- ⚠️ **不要插到本板的** `J7` —— 本板 `J7` 是 `RGB_TFTLCD` 液晶接口； 本板也**没有** `J11`。厂商 `.xls` 里的 "J6/J7/J11" 是另一块板的扩展口（详见 `docs/04` §0.1）。

1.2 拨两个电压跳线（★ 最容易漏掉、后果最严重）

跳线	选的电源轨	必须拨到
`J10`	`VCCIO_B16`（载板所在的 Bank 16）	**3.3 V**
`J9`	`VCCIO_B15`（输出侧所在的 Bank 15）	**3.3 V**

这两个 bank 的 VCCO 由可调 LDO `U22`（`SPX3819M5-L-TR`）经跳线选择，**出厂不保证 3.3 V**（丝印图例：`3.3V R1=79K`）。若不拨到 3.3 V， `LVCMOS33` 与真实 VCCO 不符， 轻则 DRC 报错、重则损伤 IO。

1.3 接信号源（★ 双端输出信号源的接法已在此逐项写清）

先看清载板这一路的拓扑，接线就不会错：

```
SMA 芯线 ──→ R13/R5 4.02K ─┬─→ AD8065（×1/5.02 缓冲）──→ AD8138（转差分）──→ AD9238 模拟入
                           └─ R15/R8 1K ──→ 地
SMA 屏蔽 ──→ 载板地
```

要点有三条：

1. **每个 SMA 通道本来就是「单端输入」**（`_refs/vendor/HSTAD9238_module_manual.pdf` p6 §2 原文： `单端输入 AD1 和 AD2 通过 J1(CH1) 或 J2(CH2) 两个 SMA 头输入，单端输入的电压为-5V~+5V`）。 芯片手册里那句"AD8138 差分输出"说的是**载板内部**把单端转成了差分再喂给 AD9238， **不是**要求你从外面送差分信号进来。
2. **输入范围 ±5 V、要求 0 V 直流偏置**（`Vout=(1.0/5.02)*vin`）。⇒ 每路设 **10 Vpp**。 若把两路接成"真差分"（一端 10 Vpp、另一端反相），等效输入摆幅会变成 20 Vpp，直接切顶。
3. **两个 SMA 的屏蔽都是载板地**，而你的信号源两路是**共地**的 （你已确认："两路独立单端输出（A/B 通道各一路，共地）"）。 ⇒ 芯线对芯线、屏蔽对屏蔽，**不存在把负端短到地的问题**，这一版接线最省事、也最安全。

接线表（按你的信号源规格：两路独立单端、共地）

信号源端	接到	电缆/转接	备注
CH1 芯线（中心针）	载板 `J1` / `CH1` 的 SMA 芯线	BNC 公 → SMA 公 50 Ω 同轴线 1 根	A 通道（RTL 里 `s_n`，正弦）
CH1 屏蔽（外壳）	载板 `J1` 的 SMA 外壳 / 载板 GND	同上，同轴线自带	与 FPGA 板共地
CH2 芯线	载板 `J2` / `CH2` 的 SMA 芯线	第 2 根同轴线	B 通道（RTL 里 `c_n`，余弦）
CH2 屏蔽	载板 `J2` 的 SMA 外壳 / 载板 GND	同上	两条屏蔽都落在同一个载板地上，无环流顾虑
（可选）外触发	载板 GND + 示波器 EXT	—	只要两路同源同频，**不是必需**

> ★ **不要用普通杜邦线接 SMA。**模拟通道虽然只跑 1 Hz 左右，但 AD8065 的输入端 对 4.02K/1K 分压点的容性和串扰很敏感；请用 **50 Ω 同轴 + BNC/SMA 转接。** 若手头只有示波器探头式的 BNC 转接，也能用（高阻输入，4.02K 分压点几乎不受影响）。

信号源参数（两路分别设，★ 两路都要设）

参数	值	说明
波形	正弦	方波也能跑，但李萨如图会变成方形，不便判读
幅度	**10 Vpp**（高阻显示）	对应 ±5 V，载板 1 LSB ≈ 2.44 mV，12 bit 满量程
直流偏置	**0 V**	载板是 AC 耦合到 ±1 V 中点，偏置不为 0 会单边切顶
输出阻抗/负载	**高阻（High-Z）**	若信号源只有 50 Ω 档，幅度会减半，请按 20 Vpp 设定或改回高阻
频率	**起始 1 Hz**	慢扫便于用 UART/示波器逐项核对；熟练后可用 `docs/04` §3.2 的慢扫流程
两路相位差	**90°**（CH2 相对 CH1）	你已确认需要 90°；这是 RTL 的 `s=sin / c=cos` 默认假设
两路耦合	**DC**（若源支持）	保证 0 V 偏置真的是 0 V；若只有 AC 耦合，先确认无下垂
两路同步	同一源的两路，或两源共 10 MHz 参考	★ 相位漂移会让李萨如图慢慢转圈，看到"转圈"先查同步

> ★ **相位方向怎么定**：`s_n=sin`、`c_n=cos` 对应 `c` 超前 `s` 90°。 若接成 CH2 滞后 90°（= `s` 超前 `c`），李萨如图仍是一个圆，但**旋转方向相反**， 表现为 `dir` 输出反向、`cycle_cnt` 递减。两种都不算故障，只要在 §8 的判定里记下方向即可 （也可用 SPI 写 `0x0109` 的 `uvwdir` / `abzdir` 位纠正）。

起始频率建议 1 Hz（`docs/04` §3.2 第 5–8 步的"慢扫"思路）。原因见 §4.1。

上电前后各做一次的两项确认

- [ ] 万用表量 `J1` / `J2` 的 SMA 芯线对载板地：**信号源未接时应为 4.02K 对地的分压电阻读数**（约 4 kΩ）， 不是 0 Ω 也不是开路 —— 这能立刻发现 SMA 头虚焊或被短接。
- [ ] 信号源接上但**先不开输出**，量芯线对地电压为 **0 V**（证明共地良好、无直流偏置）。

1.4 接输出侧的示波器探针（都用 `J5`）

`J5` 针	FPGA 脚	端口	先探什么
5	`H20`	`a_out`	★ ABZ 的 A / B / Z：**必测**
6	`G20`	`b_out`	
7	`M21`	`z_out`	★ 每机械圈 1 个
8	`L21`	`u_out`	UVW：需要先发 SPI 打开（默认关）
9	`N22`	`v_out`	
10	`M22`	`w_out`	
18/19	`J21` / `K21`	`led[2]` / `led[3]`	限频激活 / 正在标定（可接外接 LED）

J5 针	FPGA 脚	端口	先探什么
20	K22	irq_n	中断（低有效）

板载还有 4 个现成的指示：（led[0] = AB21、led[1] = AB22、key_n = W9、uart_tx = W21）。

1.5 断电状态下的万用表检查（逐项打勾）

- [ ] 模块针 1（+5V）与针 2/4/5（GND）**不短路**
- [ ] J6-1 (+5V) 与 J6-2 / J6-4 (GND) 极性正确
- [ ] ★ **AD1CLK 与 AD2CLK 在载板上不相通**（本工程用 B18→AD1CLK、F16→AD2CLK 两根独立网络分别驱动；若量到相通，停下核对，**不要短接**）
- [ ] 载板没有插反

---

2. 生成并下载 bitstream（约 10 分钟）

2.1 生产版（无 ILA）

```
cd C:\Users\zhaoy\Desktop\FPGA
$env:Path = 'D:\Vivado\2026.1\Vivado\bin;' + $env:Path
$env:HOME=$PWD.Path; $env:USERPROFILE=$PWD.Path; $env:APPDATA="$PWD\.appdata"
vivado -mode batch -nojournal -source tools/impl_check.tcl
```

产出 _synth_work/enc_top.bit。跑完检查日志里这三行：

```
（2026-10-12 v12）All user specified timing constraints are met.   <- 现行口径（sys_clk WNS +0.959 / TNS
Timing constraints are not met.          <- 2026-10-10 v9 历史口径（26 条 ADC 输入负余量，根因 = 生成时钟
INFO: [Vivado 12-1842] Bitgen Completed Successfully.
BITSTREAM_WRITTEN C:/Users/zhaoy/Desktop/FPGA/_synth_work/enc_top.bit
```

> ★ **（2026-10-12 v12）现在出现 All user specified timing constraints are met. 是正常的现行期望**（sys_clk WNS **+0.959 / TNS 0.000 / 0 个失败端点 / 11275 端点**，impl_timing.rpt @2026-10-12 23:59:16）；**若看到的是 Timing constraints are not met. 与 sys_clk 的 26 个 ADC 输入端口负余量，说明你烧/跑的是 v9 或更早的位流（2026-10-10 历史口径，根因 = 生成时钟 -invert 相位不匹配）。**
>
> ★ HOME / USERPROFILE / APPDATA 三行**必须加**，否则 Vivado 报 ERROR: [Common 17-1219] Failed to create directory to save app.xml at 'C'。

2.2 调试版（带 ILA）—— 本机已可用（GOLD 许可证）

```
# ★ 必须先指向 GOLD 许可文件，否则会命中同目录下的 BASIC 许可
$env:XILINXD_LICENSE_FILE = 'D:\Vivado\Xlnx_2026_1.lic'
vivado -mode batch -nojournal -source tools/ila_debug.tcl
```

> **★ 许可证要点（踩过坑）**：`D:\Vivado` 下**同时躺着两个许可文件** —— `D:\Vivado\Xilinx.lic`（**BASIC**）和 `D:\Vivado\Xlnx_2026_1.lic`（**GOLD**，含 `INCREMENT ChipscopePro ... License_Tier:GOLD`）。`XILINXD_LICENSE_FILE` 环境变量**默认是空的**，Vivado 会按默认搜索路径命中 BASIC 那个，于是报：
>
> `ERROR: [Vivado 12-29205] 'create_debug_core' tcl command is not supported. Your current selected license is BASIC.`
>
> 显式把 `XILINXD_LICENSE_FILE` 指向 GOLD 文件后立即通过，启动时会打印 `INFO: [Common 17-3922] A valid Vivado Design Suite GOLD license has been detected.`。`tools/run_vivado.ps1` 已把这个变量写进 `-LicFile` 参数（默认就是该 GOLD 文件），并会打印 `License_Tier:GOLD` 供核对。

`tools/ila_debug.tcl` 的流程（已按 2026.1 的 API 重写为**命名多端口**）：

- `synth_design -verilog_define ENC_TOP_ILA`（宏只能这样给，`read_verilog -define` 会报 `ERROR: [Common 17-69] ... is only supported when is_compile_unit_mode is enabled`）；
- 在 `opt_design` 之后用 `get_nets -hier -filter {MARK_DEBUG==1}` 收集探针（实测 **252 条网** = 20 组信号，约 190 bit），按基名归组、`lsort -integer -decreasing` 保证 MSB..LSB，并把宽于 32 bit 的探针切片 ⇒ **（v12 现行）36 条命名探针端口，总位宽 269**（2026-10-10 16:12 轮为 34 条 / 252，属历史）；
- 顺序必须是 `opt_design → create_debug_core → get_debug_ports → connect_debug_port`（`get_debug_ports` 只枚举**已存在**的 core 端口，提前调用只会得到 `WARNING: [Vivado 12-750] There are no ChipScope debug cores in this design.`）；
- 配置属性挂在 **core** 上、不挂在 port 上：`set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila]`（对 `get_debug_ports` 设会报 `ERROR: [Vivado 12-4442] Invalid property name C_DATA_DEPTH`）；
- 建端口前要**复用 `create_debug_core` 自动预建的那个未连接 `probe0`**，否则编号整体错位、多出一个从未连接的端口 ⇒ `ERROR: [Chipscope 16-213] The debug port 'u_ila/probe34' has 1 unconnected channels`；
- 产出 `_synth_work/enc_top_ila.bit` + `_synth_work/enc_top_ila.ltx`（**`.ltx` 必须与 `.bit` 一起加载**，否则 ILA 里只看到 `probe0[xx]` 而不是信号名）。

★ 本机实测结果（2026-10-10 16:12 用当前 RTL + 当前 XDC 重建，含 UART→SPI 桥、探针位序修复与 OTR 软清位，可直接照用）：

项	实测值
探针	**34 条命名端口，总位宽 252 bit**，`C_DATA_DEPTH 1024`、`C_INPUT_PIPE_STAGES 1`
采样时钟	`u_bufg_clk/O`（`rtl/enc_top.v:106` 的 `clk_i`；窗口 1024/50 MHz = 20.48 μs）
`route_design`	`Router Completed Successfully`；0 routing errors

项	实测值
时序	**★（2026-10-12 v12）现行期望：** `All user specified timing constraints are met.` —— 生产版 `impl_timing.rpt` @2026-10-12 23:59:16：`sys_clk` **WNS +0.959 ns / TNS 0.000 / 0 个失败端点 / 11,275 端点**（WHS +0.075 / WPWS 9.500）；ILA 版 `impl_ila_timing.rpt` @2026-10-13 07:11:57 同判据行（WNS +0.332 / 0 失败 / 20,165 端点）。**若你看到下面这组 26 条 ADC 负余量的结果，说明你烧的是 v9 或更早的位流（历史）**：`Timing constraints are not met.`；`sys_clk` 域 **WNS −3.329 ns / TNS −84.745 / 26 个失败端点（全部是 ADC 输入端口）/ 18010 端点**、`WHS +0.050 / 0 失败`、`WPWS 8.870 / 0 失败（8939 端点）`；整体 `WNS −3.329 / TNS −84.745 / 26 失败 / 19161 端点`；`dbg_hub/.../TCK` 域 WNS 27.306 / 0 失败（928 端点）；`sys_clk→adc_clk`、`sys_clk→adc_clk2` 各 +6.697 ns。★ 这与生产版 2026-10-10 16:05 的 −3.352 是**同一件事**（v9 反相相位的输入约束，见 `docs/DEV_STATUS.md` §1.19），**签核一律以生产版为准，ILA 版只作调试用途**
DRC	`Checks found: 62`，**全部 Warning 且逐条** `Related violations: <none>` ⇒ 0 error / 0 critical
资源（对比生产版）	LUT **6010** → **8822**（28.89% → **42.41%**）；Reg **3857** → **8544**（9.27% → **20.54%**）；Block RAM **0** → **7.5**（15.00%）；DSP 16 → 17（18.89%）；IOB 50 不变；BUFGCTRL 1 → 2
产出	`enc_top_ila.bit`（**1,445,812 B @ 2026-10-10 16:12:01**）+ `enc_top_ila.ltx`（**67,006 B @ 16:11:50**）；`Bitstream compression saved 5971264 bits` ⇒ `17537696−5971264 = 11566432 bit = 1,445,804 B`（+8 B 头部差自洽）

> **★ 为什么必须重新生成**：旧的 `enc_top_ila.bit`（10-09 23:31:27，1,473,128 B）**早于三处功能修复**（`rtl/calib_ctrl.v` 标定窗口 off-by-one、`rtl/uart_scope.v` 组帧、`rtl/enc_top.v` 标定窗口脉冲接错）⇒ 旧文件不能用来上板。 **10-10 08:56 那一版又早于两处修复**：① `tools/ila_debug.tcl` 的**探针位序**（见下一条）， ② `0x0405[0]` 的 **OTR 软清位**（`docs/DEV_STATUS.md` §1.21）； **10-10 15:14 那一版又早于 UART→SPI 桥**（`docs/DEV_STATUS.md` §1.22）。 ⇒ **上板只能用 16:12 这一版**（它含 UART→SPI 桥：可以用板载 USB 串口发 4 个字节读写任意寄存器，见 §5.1）。 上板用法：Hardware Manager → Program Device 里 **Bitstream 填** `enc_top_ila.bit`、**Debug probes file 填** `enc_top_ila.ltx` **（两个框都要填）**， 再把 34 条探针拖进波形窗。**建议触发**：`dbg_z_window` 上升沿（每机械圈一次）。 ⚠️ **不要用** `dbg_adc_valid` —— 它在非复位期间**恒为 1**（`rtl/ad9238_capture.v:163` `adc_valid <= 1'b1;`），**不是**"每 4 拍一个脉冲"；每 4 拍一个 80 ns 脉冲的是 `dbg_dec_valid`（probe13）。 用 `dbg_adc_valid` 做上升沿触发永远不触发，做电平触发则每次武装都在同一秒停（2026-10-10 实测）。 ⚠️ **08:56 那版的探针是位序反转的**：`tools/ila_debug.tcl` 把 net 列表按 MSB→LSB 传给 `connect_debug_port`，而**列表第一个 net 会落到探针的最低有效位** ⇒ 整条多比特探针的值被位序反转（症状：`dbg_pole_pairs_eff` 显示 8，而寄存器 `0x0100` 配置是 4；`dbg_stat_byte` 显示 0x80 而 bit7 硬接 0）。已修（`tools/ila_debug.tcl:126-137`，改成 LSB→MSB）， **15:14 这一版的** `.ltx` **里每条多比特探针的** `subnets` **顺序已整体翻转** —— 上板应看到 `4`。 若你手里只有旧位流，读 CSV 必须逐列 `bitrev` 才能读对（`tools/ila_report.py` 已内置）。 探针清单导出在 `_synth_work/ila_probe_list.txt`（`端口名 | 信号名 | 位宽`）。 ★ 时序签核一律以生产版 `_synth_work/impl_timing.rpt` 为准（ILA 版含 26 条 ADC 输入负余量端点， 与生产版同源，见 `docs/DEV_STATUS.md` §1.19）。

> ⚠️ **沙箱注意**：Vivado 生成 ILA IP 时会**派生一个子 vivado 进程并通过管道收集它的输出**（临时工程 `.Xil/.../prj_ip.runs/u_ila_synth_1/runme.log` 只有两行就再无输出，父进程无限期等待）。 受限沙箱下程序不能使用具名管道，这一步会**无限挂死**。本工程运行时对 `ila_debug.tcl` 单独放宽一次权限即可；生产版 `tools/impl_check.tcl` 不生成 IP、无嵌套子进程，不需要放宽。
>
> **也可以用 UART 遥测替代（本指南的日常手段）**：`rtl/uart_scope.v` —— 2 Mbaud、18 字节定长帧、10 kHz 帧率，帧里带 `angle` / 校正后 `s_n`、`c_n` / `amp_sq` / `pos_full` / `stat`。 用 `tools/uart_scope.py` 就能实时看波形与李萨如图，足以覆盖全部验收项（见 §7）。 ILA 的用途是抓**边沿级**细节（`dbg_a_out` / `dbg_z_out` / `dbg_edge_cnt`）。

2.3 下载

1. USB 线接开发板 `USB_UART` / `JTAG` 口，上电。
2. Vivado → `Open Hardware Manager` → `Open Target` → `Auto Connect`。
3. `Program Device` → 选 `_synth_work/enc_top.bit`。

> 若下载到一半卡死/掉线，把 JTAG 频率降到 ≤10 MHz 再试。

---

3. 第一层验证：不接信号源也该看到的（上电 5 分钟内）

#	观察点	期望现象	不符时的原因
3.1	上电后等 **≥ 21 ms**	所有输出开始活动	POR 是自由计数器，要数满 2^20 拍才释放复位。**上电瞬间没反应是正常的**
3.2	示波器探 `J6-26`（`ad_clk`）	**50 MHz**、50% 占空、3.3 V 摆幅方波	没波形 → 查 `J10` 跳线、查 bit 是否下载成功
3.3	示波器探 `J6-9`（`ad_clk2`）	**同频且与 `ad_clk` 同相**	两路反相 → 查是否只约束了一路（本版两路都有 `create_generated_clock`）
3.4	`led[0]`（`AB21`）	灭（无超量程）	常亮 → 输入超过 ±5 V，或某根数据线接错
3.5	`led[1]`（`AB22`）	灭（幅值正常）	常亮 → 幅度不是 10 Vpp / 没接信号
3.6	`led[2]`（`J5-18`）	灭 = 不限频；**接信号后可能亮**	亮 = AB 边沿被限频器压住（见 §4.1，属预期）
3.7	`led[3]`（`J5-19`）	灭	亮 = 正在标定

> `ad_clk` 是 50 MHz—— `ODDR` 用 `D1=0,D2=1` 把 `clk` 反相 180° 后转发， 所以 ADC 是 **50 MSPS**（再经 `LOG2DEC=2` 抽取到 12.5 MSPS 进流水线）。

---

4. 第二层：用 UART 遥测 + 示波器看内部行为（核心步骤）

> **为什么以 UART 为主**：UART 帧率 10 kHz、连续、不需要 JTAG 交互； ILA 是 1024 点深度的**快照**，适合抓边沿级细节。两者互补，ILA 已可用（§2.2）。 但 `rtl/uart_scope.v` 已经把最重要的内部量全部搬到了串口上， 而且**帧率 10 kHz**（50 MHz / `SAMPLE_DIV=5000`）比 ILA 的 1024 点快照连续得多。

4.1 先搞清楚默认配置下会发生什么（★ 很重要，不然会误判）

`reg_file` 复位默认值：

寄存器	默认	效果
`0x0100` UVW_CFG	`0x04`	极对数 4
`0x0101/02` INTER0/1	`0x00/0x00`	**细分 `inter=256`，分频 `div=1`**
`0x0108` ZERO1	**`0x04`**	Z 脉冲宽 4 个 **AB 边沿步**（★不是 50 MHz 拍：1 kHz/inter=256/div=1 时 1 步 977 ns ⇒ 3.9 µs，见 DEV_STATUS §1.26）；**上电即有 Z**（本版已修正，之前是 0=永不输出）。 ⚠️ **v7 及以前的位流实测只有 2 clk = 40 ns**（Z 脉宽被 `z_window` 门控窗截断， `z_width` 失效），v8 起才达到 3.9 µs —— 见 DEV_STATUS §1.26 订正块 / §1.44
`0x0104` UVW_HYST	`0x00`	**0 = 回退编译期默认档**（`UVW_HYST_SHIFT=6` → 死区 1024 LSB = 5.625° 电角度）；写 1…31 则**直接作为角度 LSB 死区**（1 LSB = 0.0054932°，最大 31 LSB = 0.170°）
`0x0109` OUTPUT	`0x40`	`abzen=1`， **`uwven=0`（UVW 三相恒 0）**
`0x0106` ABLIMIT	`0x00`	AB 限频阈值 0（最松）

推论：

- **ABZ 上电就有输出**，但在 `inter=256 / div=1` 下，一个输入电周期会产生 `4·inter/div = 1024` 个 A 边沿。1 Hz 输入 → A 边沿 1024 Hz；10 Hz 输入 → 10.24 kHz；1 kHz → 1.024 MHz。
- 一旦输入频率较高， `abz_gen` 的限频器会把边沿率压到 **1 边沿/拍 = 50 MHz 上限**， 这就是 `led[2]` 亮的原因。**属预期行为，不是故障**（`docs/04` §1.1 与 `rtl/SPEC_interface_contract.md:372`）。
- **UVW 恒 0**，因为 `uwven=0`。要测 UVW 必须先发 SPI（见 §5）。

4.2 UART 帧里有什么（这就是你的"ILA"）

`rtl/uart_scope.v:199-269` 的 18 字节定长帧（**大端**，2026-10-12 起为**帧布局 v2**）：

偏移	长度	字段	对应原来 ILA 里的哪条探针
0/1	1+1	`0xAA` `0x55` 帧头	—
2	2	`angle[15:0]`	`dbg_angle`（环路后电角度）
4	2	`s_n[15:0]`（有符号，Q1.15）	`dbg_s_cor`（**李萨如图就用它**）
6	2	`c_n[15:0]`（有符号，Q1.15）	`dbg_c_cor`
8	1	`io_state[7:0]`：bit0..5 = `a_out b_out z_out u_out v_out w_out`，bit6 = `dir`，bit7 = `limiter_active`	`dbg_a_out` / `dbg_b_out` / `dbg_z_out` / `dbg_u_out` / `dbg_v_out` / `dbg_w_out`、`dbg_dir`、`dbg_limiter_active`
9	1	`0xA2` = **帧布局标记**（v12 起；旧版此处是 `amp_sq[23:16]`，恒 ≤ `0x80` 不会撞车）	—
10	1	`io_flags[7:0]`：bit0 `lag_warn` bit1 `lag_fatal` bit2 `z_window` bit3 `uvw_freq_warn` bit4 `abzen` bit5 `uwven` bit6 `cor_sat` bit7 `amp_bad`	对应告警/使能位
11	1	`0x00` 保留（恒 0，参与校验和）	—
12	4	`pos_full[31:0]`	`dbg_pos_full`（多周期位置）
16	1	`stat[7:0]`	`dbg_stat_byte`
17	1	XOR 校验 = `0xAA ^ 0x55 ^`（偏移 2..16 逐字节 XOR）—— **含帧头常量**	—

> `amp_sq` **为什么不在帧里了**：`amp_sq = s_n² + c_n²` 由同一帧的 `s_n` / `c_n` 唯一确定（`rtl/amp_monitor.v:90-107` 只在 $2^{31}-1$ 处饱和，而 $|s|,|c| \le 32768$ ⇒ 永不触发），所以把这 4 字节让给 ABZ/UVW 电平是**无损**的：`tools/uart_scope.py` 逐帧用 `(s*s + c*c) & 0xFFFFFFFF` 重算，`amp_sq` 照旧出现在解析结果、CSV 与报告里。 0xA2 这个标记字节是新旧布局的无歧义判别位，工具遇到旧位流帧会按 v1 解释并提示。

ILA 里能看、UART 里看不到的只剩 `s_raw` / `c_raw`（ADC 原始码，用示波器量载板数据脚） 与各种 `valid` 脉冲。`a/b/z/u/v/w` 现在是**每帧一个电平快照（10 kHz）**： 边沿率高于 5 kHz 时按时间轴看会混叠，正确看法是用 `uart_scope.py --png` 第 3 行的 **等效采样图**（按电角度分箱重建波形形状），或直接示波器量 `J5` 的输出脚。

> 探针定义仍然完整保留在 `rtl/enc_top.v` 的 `` `ifdef ENC_TOP_ILA `` 段里（20 组、约 190 bit），`tools/ila_debug.tcl` 跑一遍会把清单导出到 `_synth_work/ila_probe_list.txt`。 **调试版现在就能出**（GOLD 许可证，见 §2.2）： `_synth_work/enc_top_ila.bit` 与 `.ltx` 已实测生成。

4.3 三层必做的观测

观测 A —— ADC 是否真的在采（UART）

- 信号源给 1 Hz / 10 Vpp 正弦 + 90° 相差
- `python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --csv adc.csv --png adc.png`

- 期望：`s_n` / `c_n` 是两条**相位差 90° 的正弦**，幅度接近 ±32767 的 60%~100% （10 Vpp 对应满量程，校正后仍在 Q1.15）
- 不符：
- 两条都是常数 → `ad_clk` 没到载板（回 §3.2）
- 幅度只有几百 → 信号源不是 10 Vpp
- 波形像正弦但某一位恒定 → 该数据线没接/接错，用 §4.4 的逐位法

观测 B —— 角度是否线性（UART）

- 同一份 CSV 里看 `angle`：应在一个电周期内**单调 0 → 65535 再回 0**
- `pos_full` 应按"每圈 `POLE_BITS` 个电周期"阶梯上升
- 不符：角度在 ±180° 附近跳变 → 象限折叠（本版已由 `tb_enc_top` 262140 点比对覆盖）

观测 C —— ABZ 与 Z 的关系（示波器，`J5-5/6/7`）

- 探 `J5-5`(`a_out`)、`J5-6`(`b_out`)、`J5-7`(`z_out`)
- 期望：A/B 正交方波；`z_out` 每 **64 个输入周期** 1 个（= f_in/64，1 kHz ⇒ 15.625 Hz、10 kHz ⇒ 156.25 Hz）、宽 = `zwidth`(= `0x0108`) 个 **AB 边沿步** = **`z_width / (1024 × f_in)`**（默认 4；1 kHz ⇒ 3.9 µs → 10 kHz ⇒ 0.39 µs）
- **Z 太窄看不清时**：把宽度写大 —— 发 `0x0108 = 0x400F`（`zwidth`=15 ⇒ 1 kHz 14.6 µs / 10 kHz 1.46 µs）。⚠ **不要写 `0x000F`**：包模式一次事务写 2 字节，高字节会落到 `0x0109` 把 `abzen` 清掉，Z 直接没输出；量完记得写回 `0x4004`
- ★ 2026-10-11 已在 v8 位流上实测通过：`J5-7` Z 率 **156 Hz**（10 kHz 输入）、脉宽与上式相符（见 `DEV_STATUS.md` §1.26 板上实测块）
- 不符：
- Z 完全不动 → 查 `0x0108` 是否被改成 0（默认 4）
- Z 宽到覆盖半个周期 → `zwidth` 被写大了

4.4 数据线逐位验证法（强烈推荐，能省下 3 天）

> 不要一次接 24 根线然后祈祷。

1. 信号源改成 **0.5 Hz 三角波，10 Vpp**（缓慢单调上升），接到 CH1 的 SMA 口。
2. **24 根数据线全部接好**（不要留空，输入端悬空会引入串扰）； 要定位某一位时，改用它对应的**载板数据脚**做示波器测量点（见 §1.4 的 `J6-27…` 表）。
3. 每根线用示波器看：在三角波上升沿期间，`CH1_D0` 应以**最高频率**在高低间翻转； `CH1_D1` 翻转频率是 `CH1_D0` 的 **1/2**； `CH1_Dk` 是第 0 位的 **1/2^k**。
4. 更快的方法：把三角波设到 ~1 Hz，跑 `python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --csv chk.csv`， 在 CSV 里看 `s_n` 是否从最负**单调**爬到最正再回落——中间出现"台阶"或"回跳"就说明低位错位。
5. B 通道（`CH2_D0..D11` → `J6-10..18,21..23`）同样做一遍。

---

5. 第三层：用 SPI 改配置（可选，但要让 UVW 动就必须做）

SPI 从站是 **模式 0（CPOL=0/CPHA=0，SCK 空闲低）、MSB 先、32 位包、≤10 MHz。**

32 位命令包：

```
bit31      st   = 1            （必须为 1，否则整包被忽略）
bit30:29   rm   = 00
bit28:27   wm   = 01           （寄存器写）
bit26:24   page = addr[10:8]
bit23:16   addr[7:0]
bit15:0    data[15:0]          （低字节写 addr，高字节写 addr+1）
```

常用配置（直接照抄）：

目的	写寄存器	32 位包（二进制）
打开 UVW（`uwven=1`，保留 `abzen`）	`0x0109 = 0xC0`	`1 00 01 001 00000001 0000000011000000`
把细分调到 `div=8`（AB 边沿率降到 1/8，示波器好观察）	`0x0102 = 0x1C`	`1 00 01 001 00000010 0000000000011100`
极对数改成 8	`0x0100 = 0x08`	`1 00 01 001 00000000 0000000000001000`
Z 脉宽改 15 个 AB 边沿步	`0x0108 = 0x400F`（★ 高字节 `0x40` 是给 `0x0109` 的 `abzen`，绝不能写 `0x000F`）	`1 00 01 001 00001000 0100000000001111`
一键标定（写 `CALIB_CTRL[0]`）	`0x0502 = 0x0001`	`1 00 01 101 00000010 0000000000000001`

> 没有 SPI 主机也行：**按开发板的 `KEY0`（`key_n` = `W9`）就能触发一次一键标定**（`rtl/enc_top.v` 里 `calib_ctrl.start = key_fall | calib_start_cfg`）。

5.1 没有 SPI 主机怎么办：用板载 USB 串口发 4 个字节（UART→SPI 桥，推荐）

桥（`rtl/uart_spi_bridge.v`）在 FPGA **内部**当 SPI 主机去驱动同一个 `spi_slave`，**不需要任何额外硬件**：把板载 USB 串口（CH340E）插上电脑，拿到 COM 号即可。

```
python tools/uart_spi.py --selftest                        # 不接硬件，自检编解码
python tools/uart_spi.py --port COM7 read  0x05F0          # 读 DEVICE_ID / REV
python tools/uart_spi.py --port COM7 read  0x053A          # 读 STAT_FLAGS
python tools/uart_spi.py --port COM7 write 0x0109 0x00C0   # abzen | uwven（开 UVW）
python tools/uart_spi.py --port COM7 write 0x0405 0x0001   # 清 ADC sticky OTR
python tools/uart_spi.py --port COM7 write 0x0502 0x0001   # 一键标定
python tools/uart_spi.py --port COM7 dump                  # 一次读一批常用寄存器
```

手工敲（任何串口终端：**2 Mbaud、8N1**，收发都按 HEX 显示）就是这 4 个字节：

目的	4 字节命令（HEX，MSB 先）	典型应答（6 字节）
写 `0x0109 = 0x00C0`（开 UVW）	`89 09 00 C0`	`5A … chk`
写 `0x0405 = 0x0001`（清 OTR）	`8C 05 00 01`	`5A … chk`

目的	4 字节命令（HEX，MSB 先）	典型应答（6 字节）
写 `0x0502 = 0x0001`（一键标定）	`8D 02 00 01`	`5A … chk`
写 `0x0102 = 0x001C`（`div=8`）	`89 02 00 1C`	`5A … chk`
写 `0x0108 = 0x400F`（Z 宽 15 个 AB 边沿步，**同时保住 `0x0109` 的 `abzen`**）	`89 08 40 0F`	`5A … chk`
读 `0x05F0`	`85 F0 00 00`	`5A 18 F0 10 0C AE`（本机实测；第 2 字节是 `0x053A` 的值，干净时 `10`、OTR 锁存时 `18`，**不是固定 `01`**）
读 `0x053A`	`85 3A 00 00`	`5A <flags> 3A <lo> <hi> <chk>`

- 第 1 字节 = `0x80 | (wm<<3) | page`： `wm=1` 是写、 `wm=0` 是读； `page = addr[10:8]` ⇒ 读 `0x0108` 用 `0x81`、读 `0x05F0` 用 `0x85`、写 `0x0405` 用 `0x8C`。
- 应答 = `5A | resp[31:24] | resp[23:16] | resp[15:8] | resp[7:0] | chk`， `chk = 0x5A ^ 前 4 字节`；读到的 16 位值 = `resp[15:8]<<8 | resp[7:0]`， 而 `resp[31:24]` 是地址 `{page,0x3A}` 的读值（page 5 即 `0x053A STAT_FLAGS`）。
- ★ **必须"收到应答再发下一条"**：桥一次只处理一条命令，且首字节前要求线路空闲 ≥ 13 位。
- ★ 写命令的应答内容是**上一帧**的（应答滞后一帧的重叠式），没有语义，只表示"链路通、已执行"。
- ★ 遥测（10 kHz、18 字节帧 `AA 55 …`）与应答共用一根 TX：工具会自动跳过遥测帧； 手工看波形时，`AA 55` 开头 = 遥测帧、 `5A` 开头 = 应答。
- 需要 `pyserial`（`pip install pyserial`）； `--selftest` 与 `--file` 不需要它。 （2026-10-10：已在本机装好 **pyserial 3.5**，装在用户站点目录 `C:\Users\zhaoy\AppData\Roaming\Python\Python313\site-packages`； `--selftest` = 15 checks PASS。）
- ★ **本机实测串口是 `COM5`**（`python tools\uart_spi.py ports` 会列出 `COM5 USB-SERIAL CH340`； 同时出现的 `COM3 USB 串行设备` 是幽灵设备，打不开，别选）。
- ★ **2 Mbaud 下偶发无应答**（实测约 15 条命令里 1–2 条，随机分布，与遥测帧相位无关）： 工具默认 `--retries 3` 会自动重发并成功；超时还会打印"已收字节尾部"， 用来区分"完全没应答"（位流/串口/桥没跑）与"应答被截断"（波特率/线路问题）。
- ★ 想确认一条写命令真的写进去了，加 `--verify`（写后自动回读比对）： `python tools\uart_spi.py --port COM5 --verify write 0x0100 0x0004`。 对 W1C/自清位寄存器（`0x0405`、 `0x0502`）会报"不符"，那是预期语义，不是失败。
- ★ **清 OTR 时注意幅度前提**：本模块 **10 Vpp 就是满量程**，此时 ADC 的 OTR 每个周期都真越限， 写 `0x0405=0x0001` 会毫秒级复亮；把信号源降到 **8 Vpp** 再清，就能保持（2026-10-10 实测 5 轮 ×1 s 恒 `0x0000`，见 `docs/DEV_STATUS.md` §1.23）。
- 桥与外部 SPI 主机互斥： `spi_ss_n` 被外部拉低期间桥不发帧，等待超过 2 ms 就放弃这条命令 （不写、不应答），上位机重发即可。

推荐的一套"好观察"配置（信号源 0.5 Hz）：

```
0x0102 = 0x001C   → div = 8      → A 边沿 = 4·256/8 = 128 个/电周期 = 64 Hz
0x0109 = 0x00C0   → abzen | uwven → UVW 开始换向
```

此时示波器上 `a_out` 约 64 Hz、 `u/v/w` 每相 50 % 占空、三相互差 120°。

---

6. 状态字节怎么读

`dbg_stat_byte` / UART 帧里的 `stat` / SPI 读 `0x053A` 都是同一个字（统一）：

bit	名字	=1 表示
0	`oflow`	ADC 超量程（`cor_sat` 或 `otr_lat`）
1	`falarm`	输入频率告警（本版恒 0）
2	`laglim`	滤波器滞后 或 AB 限频滞后
3	`inclim`	AB 限频激活 或 UVW 基波 > 8.33 MHz
4	`lagfatl`	致命滞后
5	`scamp`	幅值越限（`amp_low` / `amp_high`）
6	`adapt`	自适应状态（本版恒 0）
7	`res`	保留（恒 0）

正常静止/匀速测试时 `stat` 应该是 `0x00`，只有 `led[2]`（限频）那一位对应 `bit3` 可能因高细分而置起。

---

7. 量化测量层：UART 遥测上位机（最常用的工具）

> 这一层在流程上**可以随时回头做**：§4 的观测 A/B 就是用它， §8 的李萨如图验收也用它，§6 的 `stat` 状态字同样来自这里。 单独成节是因为它是整个上板过程中**唯一能给出数值曲线**的手段。

`uart_scope.v` 会以 **2 Mbaud / 8N1** 连续发 18 字节定长帧（默认 10 kHz 帧率； 50 MHz / `SAMPLE_DIV=5000`）：

偏移	长度	内容
0/1	1+1	`0xAA` `0x55` 帧头
2	2	`angle[15:0]`（0..65535 ↔ 0..360°，大端）
4	2	`s_n[15:0]`（校正后正弦，Q1.15 有符号）
6	2	`c_n[15:0]`（校正后余弦，Q1.15 有符号）
8	1	`io_state[7:0]`：bit0..5 = `a b z u v w` 电平，bit6 `dir`，bit7 `limiter_active`（**帧布局 v2**）
9	1	`0xA2` 帧布局标记（旧位流此处是 `amp_sq[23:16]`）
10	1	`io_flags[7:0]`：bit0 `lag_warn` bit1 `lag_fatal` bit2 `z_window` bit3 `uvw_freq_warn` bit4 `abzen` bit5 `uwven` bit6 `cor_sat` bit7 `amp_bad`
11	1	`0x00` 保留（参与校验和）
12	4	`pos_full[31:0]`（多周期计数<<16 | 角度）
16	1	`stat[7:0]`
17	1	XOR 校验 = `0xAA ^ 0x55 ^`（偏移 2..16 逐字节 XOR）—— **含帧头常量**

> `amp_sq` 自 v12 起**不在帧里**（由 `s_n`/`c_n` 无损重算，工具里照旧有这一列）， 腾出的 4 字节给了 ABZ/UVW 电平 + 告警位。
>
> **★ 校验和约定（2026-10-10 定案，务必不要再改回去）**：RTL （`rtl/uart_scope.v:219-224`）把两个帧头常量也异或进 `l_chk`，所以 `chk = payload_xor ^ 0xFF`。`tools/uart_scope.py` 原来只异或 payload(2..16)， 与 RTL 恒差 `0xFF` ⇒ **真机每一帧都会被判 `crc_err`、`frames` 恒为 0** （而 `--selftest` 因为自己组帧自己解析，照旧 PASS，发现不了）。 定案取 RTL 一方（位流 10-10 02:32 / WNS +0.686 ns 已按此实现并验证， 为一条常量偏移改 RTL 会作废整个时序收敛结果）。 守卫：`sim/fixtures/uart_frame_rtl.hex` （从 `sim/tb_uart_scope.v` 的 `RTL_FRAME_HEX` 行实拍的 18 字节）+ `python tools/check_uart_frame_fixture.py`。

用法：

```
python tools\uart_scope.py --selftest                          # 不接硬件，先验证解析逻辑
python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 10 --csv out.csv
python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5  --csv adc.csv --png adc.png
```

- 第一件事：用 `--selftest` 确认工具本身没问题（它不依赖硬件）。
- 第二件事：确认 COM 口存在、波特率设成 **2000000** （USB 串口驱动要支持 2 Mbaud；若上位机只能到 921600，就把 `rtl/enc_top.v` 的 `BAUD_DIV` 从 25 改成 54 重新生成 bitstream，同时把命令里的 `--baud` 改掉）。
- 收不到任何字节 → 检查帧头 `0xAA55` 是否出现在 CSV 里； 若全无数据，先确认 `uart_tx`(`W21`) 就是板载 USB 串口的那一根 （它是板载固定的，不在 `J5`/`J6` 上）。
- `--png` 会画出 `s_n-c_n` 的**李萨如图**——这就是 §8 第 8 条验收项的直接依据。

---

8. 验收判定标准（做到这几条就算"上板通过"）

#	判定项	通过标准	怎么测
1	时钟	`ad_clk` 与 `ad_clk2` 都是 50 MHz、同相、3.3 V 摆幅	示波器探 `J6-26` / `J6-9`
2	ADC 数据	`s_n`/`c_n` 是相位差 90° 的正弦，幅度接近 Q1.15 满量程	`uart_scope.py --csv` （对应原来的 `dbg_s_raw`/`dbg_c_raw`）
3	逐位正确	§4.4 的"第 k 位翻转频率 = 第 0 位 1/2^k"全部成立	示波器逐脚；或用 `s_n` 三角波单调性快查
4	角度线性	UART 的 `angle` 在一个电周期内单调 0→65535， `pos_full` 阶梯上升	`uart_scope.py --csv`

#	判定项	通过标准	怎么测
5	Z	每 **64 个输入周期** 1 个 Z（= f_in/64；1 kHz ⇒ 15.625 Hz，10 kHz ⇒ 156.25 Hz）；宽 = **`(z_width + 1)` 个 AB 边沿步 = `(z_width + 1) / (1024 × f_in)`**（★ v9 起按 公开手册口径；v8 及以前少 1 步。1 kHz + 默认 4 ⇒ **4.88 µs**（v8：3.9 µs）；10 kHz + 4 ⇒ 0.49 µs）。★ 想把脉冲拉宽好看：写 **`0x0108 = 0x400F`**（`z_width` = 15 ⇒ 16 步：1 kHz 15.6 µs / 10 kHz 1.56 µs），量完写回 `0x4004` —— ⚠ **不要写 `0x000F`**：包模式一次写 2 字节，高字节会落到 `0x0109` 把 `abzen` 清掉，Z 就没输出了（`rtl/reg_file.v:565,577`）。⚠ **v9 起 `z_width = 0` 不再是"无 Z"，而是 1 个边沿步**（977 ns @1 kHz）	示波器探 `J5-7`（`docs/DEV_STATUS.md` §1.26 板上实测块：2026-10-11 **v8 位流**实测 **Z 率 156 Hz + 脉宽相符**——注意那次读数对应 v8 的 `z_width` 步语义；同日 **v9 位流**已用 ILA（`ILA_TRIG=z_out`）直测 **`z_width = 4` ⇒ 240 拍 = 5 个边沿步**、**`0x0108 = 0x4000` ⇒ 60 拍 = 1 个边沿步**，示波器旁证待补）
6	状态字	静止/匀速时 `stat = 0x00`；`led[0]`/`led[1]` 全灭	UART 第 16 字节 / 板载 LED
7	UVW（可选）	发 `0x0109=0xC0` 后，`u/v/w` 三相互差 120°、各 50 % 占空	示波器探 `J5-8/9/10`
8	李萨如图	`uart_scope.py --png` 画出的圆是正圆（跑过一键标定后更圆）	`uart_scope.py --png`

李萨如图的读法（这是最直观的验收）：

你看到的	说明
正圆、圆心在原点	无误差
圆心偏离原点	直流偏置误差（由 `OFS_S`/`OFS_C` 校正）
斜椭圆	幅值失衡或正交相位误差（由 `GAIN_S`/`GAIN_C`/`PHASE_K` 校正）
按 `KEY0` 一键标定后变正圆	**六参数闭式标定生效**

> ⚠ **A 边沿计数这一项在本机无法通过 UART 精确验收**：UART 帧里没有 `edge_cnt`，而 `a_out` 在 `inter=256/div=1` 下会跑到 1 MHz 以上。要看精确边沿数，请把 `div` 写到 8 以上（`0x0102=0x1C`）再用示波器频率计测 `J5-5`，或在有 ILA 许可证时用 `dbg_edge_cnt`。这一点已如实记入 §10。

---

9. 故障速查（上板专属）

现象	最可能原因	动作
上电后 20 ms 内完全无输出	POR（预期）	等 21 ms
`ad_clk` 无波形	bit 未下载；或 `J10` 跳线不是 3.3 V	重下 bit；拨跳线
想要 ILA 但报 `12-29205 ... license is BASIC`	`XILINXD_LICENSE_FILE` 没设，命中了同目录的 BASIC 许可	设 `$env:XILINXD_LICENSE_FILE='D:\Vivado\Xlnx_2026_1.lic'` 再跑（§2.2）；确认启动日志有 `A valid Vivado Design Suite GOLD license has been detected`
`uart_scope.py` 一个字节都收不到	COM 口不对 / 波特率不是 2000000 / 驱动不支持 2 Mbaud	`--selftest` 先排除工具问题；再把 `BAUD_DIV` 改成 54（921600）重出 bitstream
UART 帧头 `0xAA55` 时有时无	波特率偏差或线太长	换短 USB 线；确认上位机波特率精确 2000000
两条 ADC 波形一模一样（没有 90°）	信号源两个通道没设相差，或 B 通道线接错	查信号源 `CH2 PHASE = 90°`
数据随机跳变	捕获相位不对 / 线太长 / 没共地	缩短线（≤5 cm）、共地、扫相位 0/90/180/270
某一位恒定不变	该数据线没接/接错	§4.4 逐位法
`led[0]` 常亮（OTR）	输入超过 ±5 V	降到 10 Vpp
`led[1]` 常亮（幅值告警）	幅度不对或未标定	设 10 Vpp；按 `KEY0` 标定
`led[2]` 常亮（限频）	细分 `inter=256` + `div=1` 时边沿率超 50 MHz	**预期行为**；要消除就写 `0x0102=0x1C`（div=8）
UVW 一直是 0	`uwven=0`（默认关）	写 `0x0109=0xC0`
Z 一直是 0	`0x0108` 被写成 0，**或** 之前写 `0x0108 = 0x000F` 把 `0x0109` 的 `abzen` 清掉了	写 `0x0108=0x4004`（恢复 `zwidth`=4 且保住 `abzen`），再读 `0x0109` 确认 bit6 `abzen` = 1
下载 bit 后板子就"死"	JTAG 时钟太快	JTAG 频率 ≤10 MHz
报 `Bank 16 / Bank 15 VCCO 与 IOSTANDARD 不符`	`J10` / `J9` 不在 3.3 V	拨到 3.3 V
输出侧（ABZ/SPI/`led[2:3]` / `irq_n`）无波形	这些脚在 `J5` 不是 `J6`	按 §1.4 表接线

10. 已知缺口（上板时会遇到的、已在报告里如实标注的）

#	现象	性质	现状
1	~~`div=1` + `inter=256` 满分辨率时，AB 输出会静默丢 A 边沿~~	✅ **真实 RTL 缺陷，已定位并修复**（真根因是 `acc_th` 声明为无符号、与有符号的 `acc_step_sum` 混比，导致反向被误判成正向点火、累加器滚向负饱和）	**不需要再做任何回避。**修复内容：`acc_th` 改有符号 + 差路径加宽到 13 位有符号 + 删掉一行遗留的重复 `assign step_val` + 补残留进位排空；黄金模型同步。隔离台 A/B 边沿数现在**逐项精确等于** `4*inter/div`，集成台一圈 A 沿 4096 = SPEC 精确值，全量回归 18/18 PASS（`sim/logs/SUMMARY.md` 2026-10-10 07:43:53）
2	~~UVW 迟滞 `cfg_hyst_uvw` 没接到 `uvw_gen` ~~	✅ **已接通**（2026-10）：`rtl/uvw_gen.v` 新增 `input wire [4:0] uvw_hyst`，`rtl/enc_top.v` 把它接到 `cfg_hyst_uvw`（寄存器 `0x0104` 的 `[4:0]`）	语义：**`0x0104 != 0` 时该值直接作为角度 LSB 死区**（1 LSB = 0.0054932° 电角度，5 位上限 31 LSB = 0.170°）；**`0x0104 == 0x00` 时回退到 elaboration 参数 `UVW_HYST_SHIFT`**（默认 6 → 1024 LSB = 5.625°），保证未驱动该端口的旧实例行为不变。`sim/tb_uvw_gen.v` 新增 TEST 9 覆盖（含扇区等宽校验与 `hyst_lsb=1024/31` 回退校验）
3	~~`rtl/uvw_gen.v` 的 `eang*6` 因位宽不足被截断成 `eang*5` ~~	✅ **真实 RTL 缺陷，已定位并修复**（2026-10）。原式 `{eang,2'b00}+{eang,1'b0}` 两个操作数都只有 `AW+2=18` 位 ⇒ `(eang*6) mod 2^18`，`[AW+2:AW]` 取到的是 `eang*5` 的高 3 位	影响：**65536 个角度里 32768 个（50.0%）扇区号偏小 1**，真实扇区边界变成 `13108/26215/39322/52429`（应为 `10923/21846/32768/43691/54614`），且 4→5→6→7→0 有 1 拍毛刺态。修法：`assign eang_x6 = ({3'b0, eang} << 2) + ({3'b0, eang} << 1);`（19 位够容纳 `6*65535=393210`）。原 TEST 1/2 只查"6 次切换"与驻留极差 ≤2，**结构上不可能发现**；新 TEST 9 逐角度扫描 65536 点后实测 5 个边界**精确等于** `ceil(k*65536/6)`
4	`sim/tb_enc_top.v` 的"一圈"定义与 SPEC 不符	**测试台 bug，RTL 正确**（已由深度探针实测证实）	回归仍是 PASS（容差兜住），留待后续修正；TEST F 已另加顶层标定集成覆盖（63 checks）
5	~~本机没有 ILA~~ → **ILA 现已可用**	**不再是缺口**：Vivado 许可证已升级到 **GOLD**（`D:\Vivado\Xlnx_2026_1.lic`，含 `INCREMENT ChipscopePro ... License_Tier:GOLD`）。必须显式设 `XILINXD_LICENSE_FILE`，否则会命中同目录的 BASIC 许可	跑 `tools/ila_debug.tcl` 产出 `_synth_work/enc_top_ila.bit` + `.ltx`（见 §2.2）。★ 该脚本会派生带管道的子 vivado 进程，受限沙箱下会挂死，需单独放宽一次权限
6	上板时**无法用 UART 精确验收"A 边沿数"**	UART 帧里没有 `edge_cnt`；而 `inter=256/div=1` 时 `a_out` 频率 >1 MHz	把 `div` 写到 ≥8（寄存器 `0x0102` 的 `[4:2]` 是 `div-1`，写 `0x1C` 得 div=8）后用示波器频率计测 `J5-5`；或直接用 ILA 看 `dbg_edge_cnt`（调试版已可用，见 §2.2；RTL 侧该项已由 `tb_enc_top` 定量验证：正反两向各 262140 点零超差、A 沿 256 = SPEC 精确值）

附：一次完整上板的最小流程（速查卡）

1. 断电 → 载板插 J6 → J10/J9 拨 3.3V → 信号源两路(各10Vpp/0V偏置/相差90°)经 50Ω 同轴接载板 J1(CH1)/J2(CH2)
2. 万用表查极点性与 AD1CLK/AD2CLK 不相通
3. 上电 → vivado -mode batch -source tools/impl_check.tcl （生成 enc_top.bit）
4. Program Device 下 _synth_work/enc_top.bit
5. 等 21ms → 示波器看 J6-26/J6-9 = 50MHz 同相
6. 装 USB 串口驱动 → python tools\uart_scope.py --selftest
7. python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --png lis.png
   → 看 s_n/c_n 是否 90° 正弦、angle 是否单调、圆是否正圆
8. 示波器探 J5-5/6/7 → 看 A/B 正交与 Z（每圈 1 个）
9. 按 KEY0 一键标定 → 看 led[3] 亮一下，之后再抓一次李萨如图应更圆
10. （可选）SPI 写 0x0109=0xC0 → 示波器看 u/v/w 三相互差 120°